



ADVANCED JAVASCRIPT · TERRALOGIC ACADEMY

Day 2: Objects & Prototypes

Session recap for 19th July 2026 · definitions + real-world examples for topics actually covered

day2.js

```
// Run with: node day2.js  
topics = ['creation patterns', 'prototype (Object.create)', 'classes'];
```

Today's 5 concepts

- 01 Object Creation Patterns** — literals, constructors, factories, ES6 classes
- 02 Prototypal Inheritance with `Object.create()`** — linking objects to a shared prototype
- 03 `Object.assign` & `Object.freeze`** — cloning/merging and locking objects
- 04 Getters & Setters** — validating on write, computing on read
- 05 ES6 Classes & Constructors** — bundling state and behavior, instantiating with 'new'

Each concept: a clear definition, then real-world code examples — trimmed to only what was covered in today's live session.

Object Creation Patterns

DEFINITION

JavaScript objects can be created several main ways: object literals for one-off objects, constructor functions with 'new' for many similar instances sharing methods, factory functions that simply return a plain object, and ES6 classes as a cleaner syntax over constructors.

KEY POINTS

- Object literal — fastest way to make a single, unique object
- Constructor + new — instances share methods via the prototype
- Factory function — returns a plain object, no 'new' or 'this' needed
- ES6 class — same idea as a constructor, cleaner syntax

EXAMPLE 1

day2.js

```
// object literal
const user1 = { name: 'Asha', role: 'admin' };

// constructor function
function User(name, role) {
  this.name = name;
  this.role = role;
}
User.prototype.describe = function() {
  return `${this.name} (${this.role})`;
};
const user2 = new User('Ravi', 'editor');

// factory function
const createUser = (name, role) => ({ name, role });
```

Example 2 — Choosing a Pattern for Account Objects

In an auth system with thousands of user objects, a constructor keeps shared logic (like password checks) on one prototype instead of duplicating it per user, while a lightweight factory suits simple, role-based config objects.

```
accounts.js

// constructor – shared method lives once on the prototype
function Account(email, passwordHash) {
  this.email = email;
  this.passwordHash = passwordHash;
}
Account.prototype.checkPassword = function(hash) {
  return hash === this.passwordHash;
};
// 10,000 accounts share ONE checkPassword function in memory

// factory – quick, no 'new', good for simple role configs
const createGuestSession = () => ({
  role: 'guest', permissions: [], expiresIn: 3600,
});
```

Why it matters: At scale, method placement matters: prototype methods are shared once, while methods defined inside every object literal or factory are duplicated per instance.

Prototypal Inheritance with Object.create()

DEFINITION

Every object has an internal link to another object — its prototype.
Object.create(proto) builds a new object whose prototype is set directly to proto, wiring up inheritance without needing a constructor.

KEY POINTS

- Object.getPrototypeOf(obj) reveals the link
- Object.create(proto) sets an object's prototype directly
- Methods placed on a shared prototype are used by every linked object
- If a property isn't found on the object, JS looks up the chain

EXAMPLE 1

day2.js

```
const animalMethods = {  
  speak() { return `${this.name} makes a sound`; }  
};  
  
const dog = Object.create(animalMethods);  
dog.name = 'Rex';  
  
dog.speak(); // found on the linked prototype  
Object.getPrototypeOf(dog) === animalMethods; // true
```

Example 2 — Shared Behavior Across UI Components

A vanilla-JS widget library defines common behavior once on a base component prototype (logging, event binding) and links each specific widget — buttons, modals, tooltips — to it with `Object.create()`, so no code is copied per widget.

```
components.js

const componentBase = {
  log(msg) { console.log(`[${this.type}] ${msg}`); }
};

const saveBtn = Object.create(componentBase);
saveBtn.type = 'IconButton';
saveBtn.icon = 'save';

saveBtn.log('clicked'); // inherited, not duplicated
```

Why it matters: Linking objects to a shared prototype with `Object.create()` is the core mechanism behind reusing behavior across many similar objects without copy-pasting methods.

Object.assign & Object.freeze

DEFINITION

Two built-in utilities for controlling objects: `Object.assign(target, ...sources)` shallow-copies properties into a target — useful for cloning and merging — and `Object.freeze(obj)` locks an object against further changes.

KEY POINTS

- `Object.assign(target, ...src)` — merge configs or shallow-clone objects
- `Object.freeze(obj)` — makes top-level properties read-only
- Frozen $\neq$ deeply immutable — nested objects can still be mutated
- Assigning to a frozen property fails silently (or throws in strict mode)

EXAMPLE 1

day2.js

```
// merging default options with user overrides
const defaults = { theme: 'light', retries: 3 };
const userOptions = { theme: 'dark' };

const config = Object.assign({}, defaults, userOptions);
// { theme: 'dark', retries: 3 }

Object.freeze(config);
config.theme = 'blue'; // silently ignored
console.log(config.theme); // still 'dark'
```

Example 2 — Immutable App State Updates

State-management libraries never mutate state directly. Every update returns a brand-new object via `Object.assign()`, and app-wide constants are frozen at startup so no module can accidentally change shared config.

```
store.js

// frozen constants – safe to import anywhere
const APP_CONFIG = Object.freeze({
  apiVersion: 'v2', maxUploadMB: 25,
});

// reducer never mutates – returns a new object each time
function cartReducer(state, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      return Object.assign({}, state, { items: [...state.items, action.item] });
    default:
      return state;
  }
}
```

Why it matters: Redux, Zustand, and React state all lean on this 'never mutate, always return new' rule — it's what makes undo/redo and change-detection reliable.

Getters & Setters

DEFINITION

Getters and setters are special methods that run when a property is read or written, while still looking like a plain property to whoever is using the object. They let you compute values on read, or validate/react on write.

KEY POINTS

- Defined with the get / set keywords in an object literal or class
- get — compute a value on the fly instead of storing it directly
- set — validate or transform a value before it's stored
- Caller uses normal dot syntax (obj.prop) — no extra method calls

EXAMPLE 1

day2.js

```
const temperature = {
  _celsius: 0,
  get celsius() { return this._celsius; },
  set celsius(value) { this._celsius = value; },
  get fahrenheit() {
    return this._celsius * 9/5 + 32;
  },
};

temperature.celsius = 25;
temperature.fahrenheit; // 77 - computed on read
```

Example 2 — Validating Input in a Signup Form

A User class uses a setter to reject an invalid email the moment it's assigned — no separate validate() call to remember — and a getter to compute a display name on demand instead of storing a redundant field.

```
user.js

class User {
  constructor(firstName, lastName) {
    this.firstName = firstName;
    this.lastName = lastName;
  }
  set email(value) {
    if (!value.includes('@')) {
      throw new Error('Invalid email');
    }
    this._email = value;
  }
  get displayName() {
    return `${this.firstName} ${this.lastName}`;
  }
}
```

Why it matters: Catching bad data at the moment of assignment — not in a separate validation pass — is exactly how real signup and profile-edit forms stay consistent.

ES6 Classes & Constructors

DEFINITION

The class keyword is syntactic sugar over a constructor function — a cleaner way to bundle state and behavior. Instances are created with the 'new' keyword, just like constructor functions, and methods declared in the class body are placed on the prototype automatically.

KEY POINTS

- `constructor()` runs automatically when you call `new ClassName(...)`
- Methods in a class body live on the prototype, not each instance
- 'this' inside a method refers to the specific instance calling it
- A class is a cleaner syntax over the constructor-function pattern

EXAMPLE 1

day2.js

```
class Shape {  
  constructor(name) { this.name = name; }  
  describe() { return `This is a ${this.name}`; }  
}  
  
const shape1 = new Shape('circle');  
shape1.describe(); // "This is a circle"
```

Example 2 — Modeling Bank Accounts with a Class

A `BankAccount` class bundles an owner and balance together with the deposit/withdraw behavior that operates on them. Every account made with `'new BankAccount(...)'` gets its own state but shares the exact same methods.

```
bank.js

class BankAccount {
  constructor(owner, balance = 0) {
    this.owner = owner;
    this.balance = balance;
  }
  deposit(amount) {
    this.balance += amount;
    return this.balance;
  }
  withdraw(amount) {
    if (amount > this.balance) throw new Error('Insufficient funds');
    this.balance -= amount;
    return this.balance;
  }
}
```

Why it matters: Every account created with `'new BankAccount(...)'` keeps its own owner and balance while reusing the same deposit/withdraw logic — exactly how real banking and e-commerce systems keep instance data separate but share behavior.

RECAP

Day 2 in one glance

- ✓ Creation patterns: literal, constructor, factory, or class — pick by scale
- ✓ Prototypal inheritance: `Object.create()` links an object to a shared prototype
- ✓ `Object.assign` & `Object.freeze`: merge/clone objects, then lock down config
- ✓ Getters/setters: validate on write, compute on read, plain syntax
- ✓ Classes & constructors: bundle state + behavior, instantiate with 'new'

Practice: build a `BankAccount` class using a constructor, then rewrite the same thing as a factory function to compare the two patterns.